

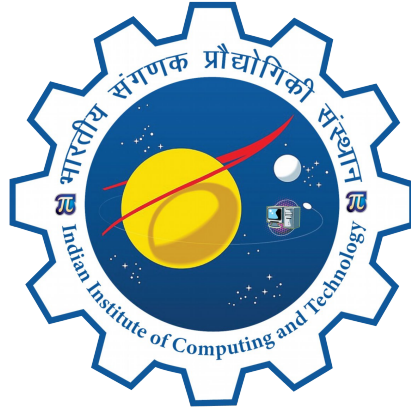


भारतीय संगणक एवं प्रौद्योगिकी संस्थान

INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

AI-Powered Fake News Detection



Project Report

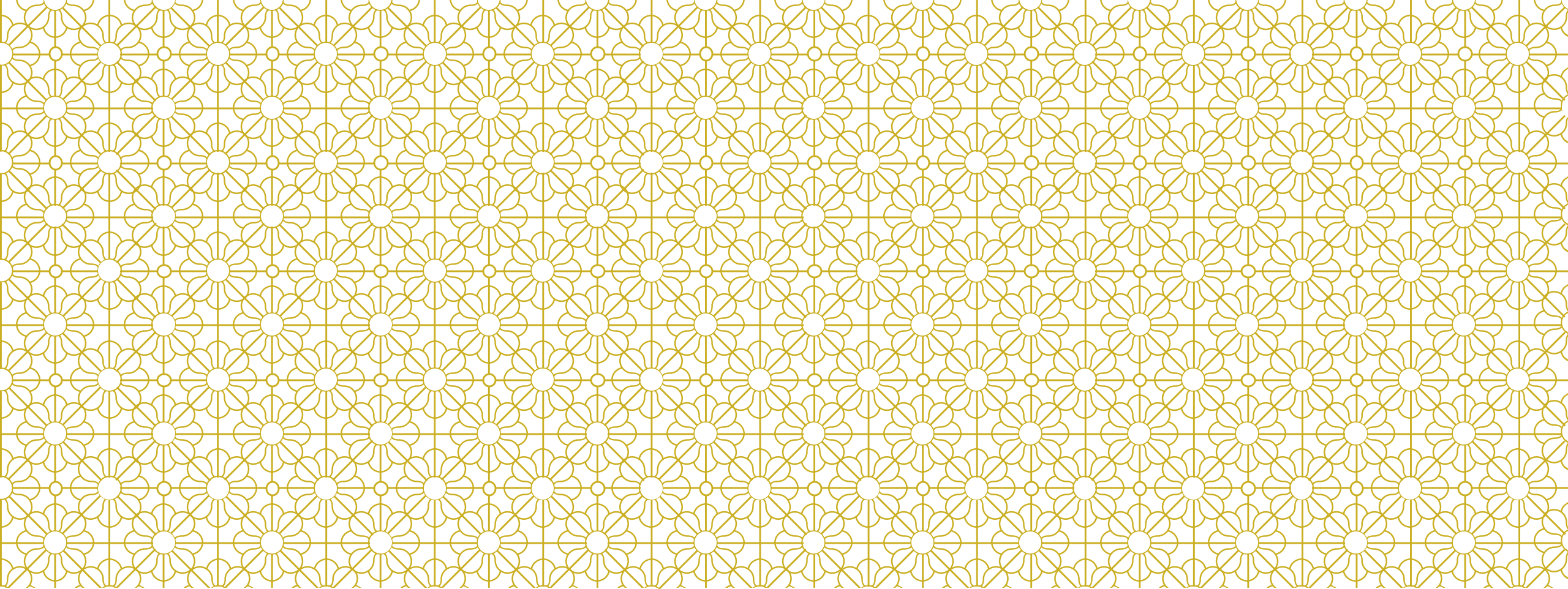
15th June to 30th July 2026

Student Name: Asmit Das Choudury

Institute Name: BrainWare University

Institute Roll no.: 24103320100

Enrollment no.: 380119



AI-POWERED FAKE NEWS DETECTION USING TEXT CLASSIFICATION

Name: Asmit Das Choudhury
Institution: Brainware University
Email: asmitdaschoudhury@proton.me
Reg. No.: 380119

TABLE OF CONTENTS

- I. Abstract
- II. System Approach
- III. Project Structure
- IV. Project Workflow
 - A. Overview
 - B. Data Cleaning and Preprocessing
 - C. Exploratory Data Analysis
 - D. Model Training and Evaluation
 - E. Deployment via Streamlit
- V. Conclusion

ABSTRACT

Social media network is considered one of the significant aspects of human life due to the recent developments of technology and computer science. This environment has become a popular place to share different types of information and news. There are a lot of pros of this system such as easy access of global news from the comfort of one's home, but from another perspective, there are also many fake information and news that mislead users about the required information. Lack of trustworthy information is the biggest problem of this environment.

Hence, to overcome this problem, I have proposed an AI-Powered system, in form of a streamlit web-deployed application which utilises natural language processing (NLP) to apply machine learning techniques to detect fake news. The solution combines both classical machine learning and deep learning models, whose trainings are documented in the provided jupyter notebook.

SYSTEM APPROACH

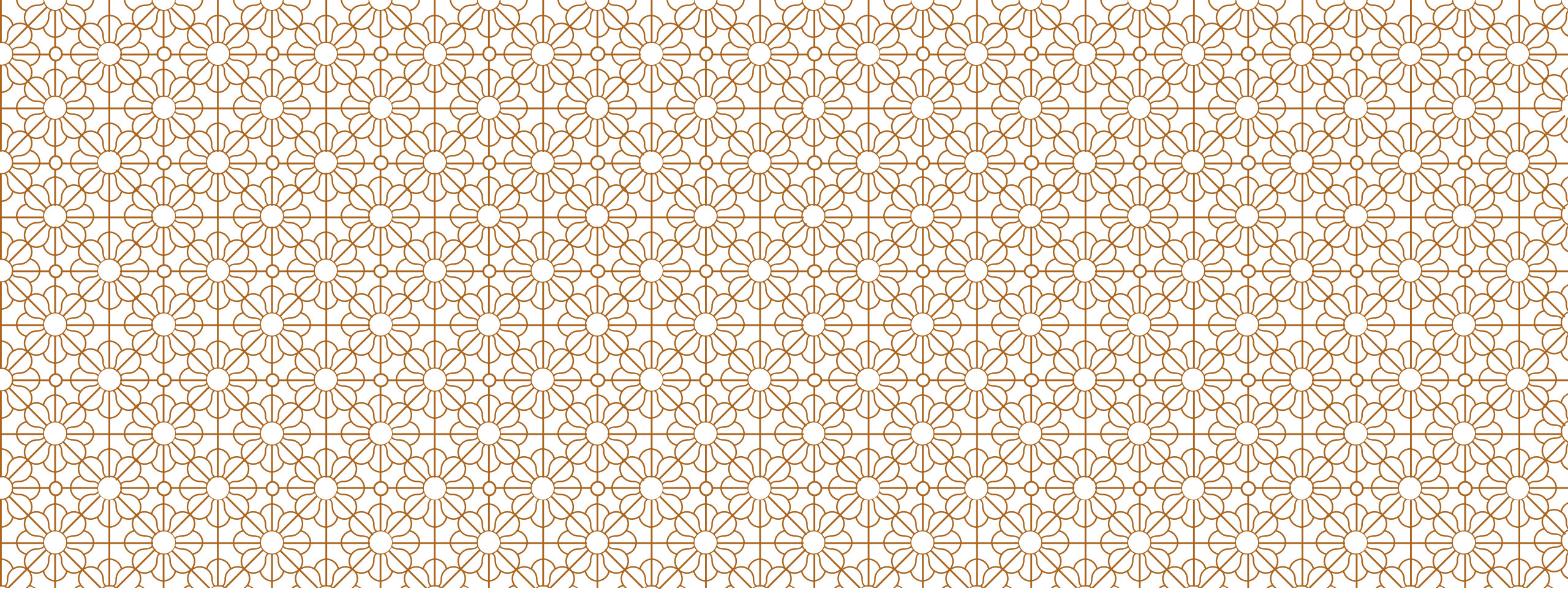
The system is organized as a staged workflow that starts with data acquisition and ends with deployment-ready model artifacts. The notebook relies on these main libraries and tools:

- **pandas** and **numpy** → for data loading, inspection, and cleanup operations
- **kagglehub** → for downloading the Kaggle dataset used in the notebook
- **matplotlib**, **seaborn**, and **wordcloud** → for exploratory visualization (EDA)
- **nltk** → for tokenization, stopword removal, and lemmatization
- **scikit-learn** → for train/test splitting, preprocessing, pipelines, feature encoding, scaling, and classical models
- **tensorflow** and **scikeras** → to support the feedforward neural network workflow
- **joblib** → for saving and loading the fitted preprocessor and trained models
- **streamlit** → for deploying prediction web-app

PROJECT STRUCTURE

The repository is split into a notebook-driven training workflow, a lightweight deployment app, saved model artifacts, and supporting data files.

- **app.py & model_pred.py** → Streamlit interface for entering article details and viewing predictions.
- **fake_news_detector.ipynb** → Jupyter notebook which documents the complete workflow from retrieving the dataset to training and evaluating the models.
- **datasets/** → Raw and cleaned dataset files used during exploration and model training.
- **models/** → Serialized preprocessor and model files used by the web application.
- **screenshots/** → Visual assets and captured screenshots for presentation or documentation.
- **others/** → Miscellaneous files such as the problem statements pdf and other documents.



PROJECT WORKFLOW

- I. Overview
- II. Data Cleaning & Preprocessing
- III. Exploratory Data Analysis
- IV. Model Training and Evaluation
- V. Deployment via Streamlit

I. OVERVIEW

The Jupyter Notebook first loads the Kaggle dataset, removes irrelevant or low-quality records, then transforms the text and structured fields into machine-readable features. After that, the data is split into training and testing sets, multiple models such as Logistic Regression, K-Nearest Neighbours, Random Forest and Feed-Forward Neural Network, are trained, and evaluation metrics are collected for comparison. And finally, the artifacts are exported which are to be used by the deployment app.

II. DATA CLEANING & PREPROCESSING

Real-world data is often incomplete, noisy, and inconsistent, which can lead to incorrect results if used directly. Data preprocessing in data mining is the process of cleaning and preparing raw and messy data. Cleaning up the data before training models on it is extremely crucial for the models to be able to process it as it intended to.

- Dropping null entries → removed **330** entries with null ``source_domain`` values
- Dropping irrelevant columns → removed the ``news_url`` from the data
- Fixing incorrect data types → no conversion was needed as they are already in correct dtypes
- Dropping duplicate entries → removed **1468** entries with duplicate ``title`` values
- Dropping outliers → removed **576** entries with ``tweet_num`` > 500 (out of **20,000** entries)
- Cleaning textual values → preprocessed the ``title`` and ``source_domain`` using **nltk** library

#	Column	Dtype	Mean	Min	Max	Description
1	title	string (object)	-	-	-	contains the title of the news article
2	source_domain	string (object)	-	-	-	contains the website where the article was posted
3	tweet_num	Int64	45.79	0.0	500.0	contains the no. of times the article was retweeted
4	real	Int64	0.77	0.0	1.0	whether the news article is real or fake

DATASET INFORMATION

Total Entries: 20822

No. of Fake News: 4607

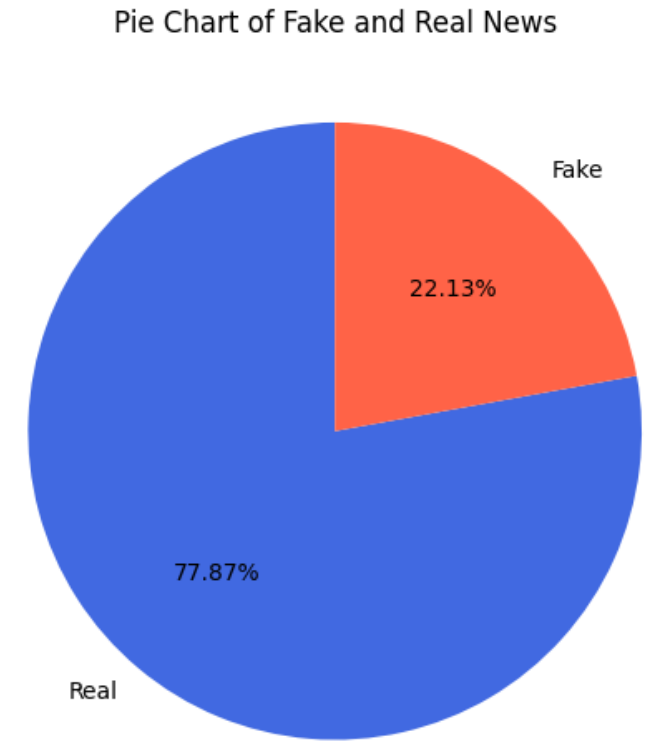
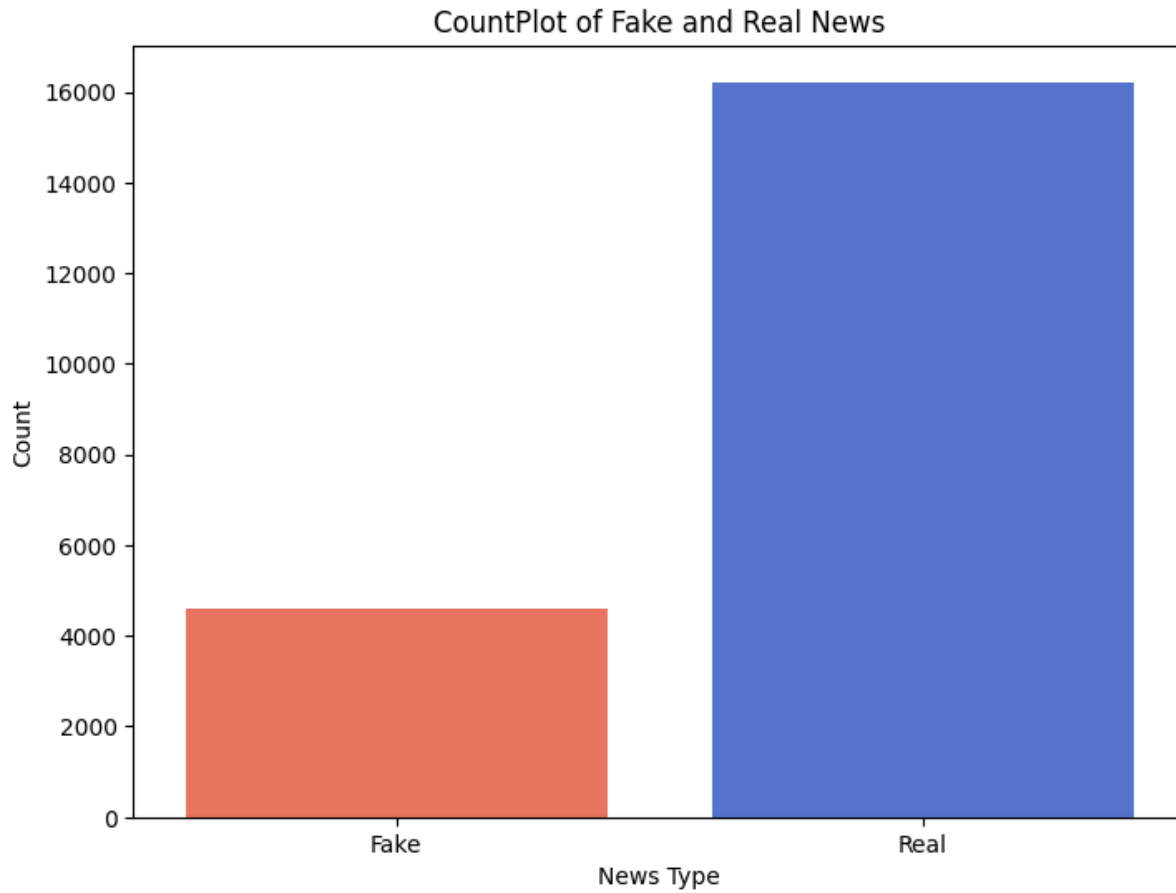
No. Of Real News: 16215

III. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis is used to understand the balance between fake and real news, inspect the distribution of retweet counts, and visualize patterns in the text data through word clouds. This step helps reveal whether the classes are imbalanced and whether the source or engagement patterns may carry predictive value.

This EDA section involves visualization of the following:

- Distribution of News Type → using Bar Graph & Pie Chart
- Distribution of Retweets → using Histogram and Box Plot (for detecting outliers)
- WordCloud of Real and Fake news
- Correlation Analysis of `tweet\_num` and `real` → using Heatmap

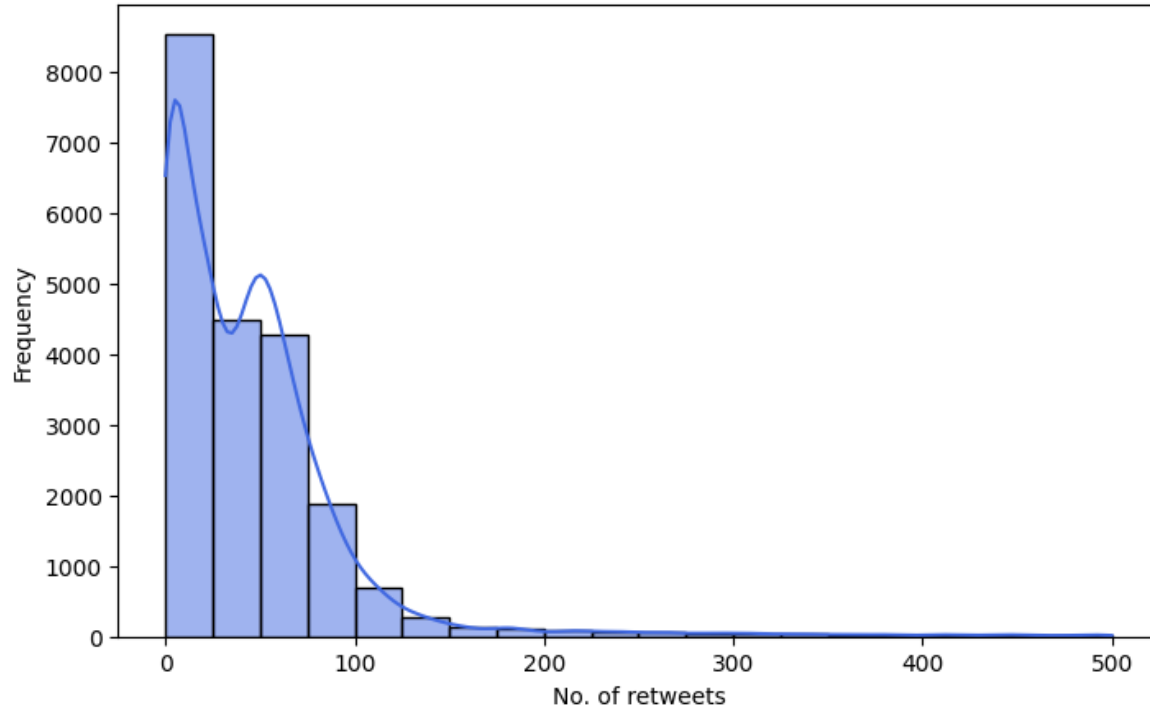


DISTRIBUTION OF NEWS TYPE

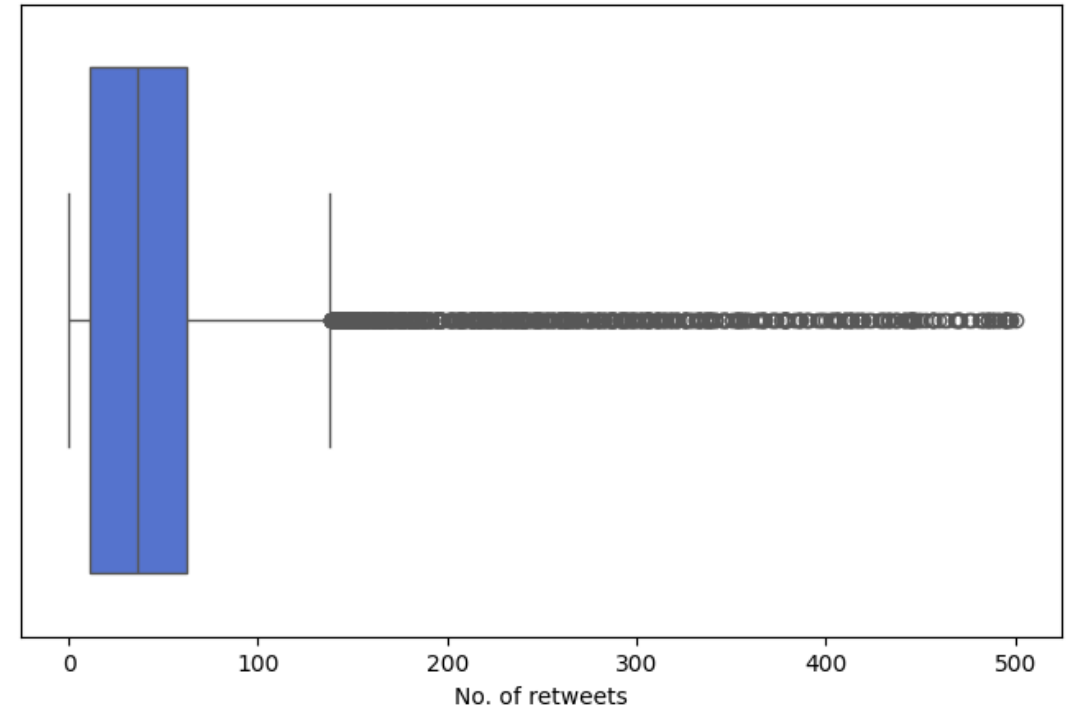
LEFT: Bar Plot (CountPlot)

RIGHT: Pie Chart

Histogram of Tweet Numbers



Box Plot of Tweet Numbers



DISTRIBUTION OF RETWEETS

LEFT: Histogram

RIGHT: Box Plot

[illegible][illegible]

LEFT: Fake News
RIGHT: Real News

IV. MODEL TRAINING AND EVALUATION

At the pipeline level, the system applies `OneHotEncoder` to the `source_domain`, `StandardScaler` to the `tweet_num` count, and `TfidfVectorizer` to the cleaned `title` text. These transformed features are then passed into the model-specific classifiers using `ColumnTransformer`, allowing the same preprocessing logic to be reused consistently across training and inference.

The following models were trained using the appropriate a parameters and then evaluated using classification reports and confusion matrix:

- Logistic Regression → **Max Iterations: 1000**
- K-Nearest Neighbours → **Nearest Neighbours (k): 5**
- Random Forest → **No of Estimators (trees): 50**
- Feed-Forward Neural Network → **Epochs: 60 and Batch Size: 24**

Model	Accuracy	Precision	Recall	F1 Score	Description
Logistic Regression	87.07%	87.51%	97.26%	92.13%	scores the highest Recall among other models
K-Nearest Neighbours	86.36%	91.82%	94.34%	93.50%	scores the highest Precision among other models
Random Forest	87.42%	87.14%	95.52%	91.75%	moderate performance despite taking long time to train
Feed-Forward Neural Network	88.36%	90.71%	94.74%	92.68%	scores the highest Accuracy among other models

MODEL EVALUATION

NOTE: the values might not match exactly with the jupyter notebook as I might rerun those in the future which will result in different values

V. DEPLOYMENT

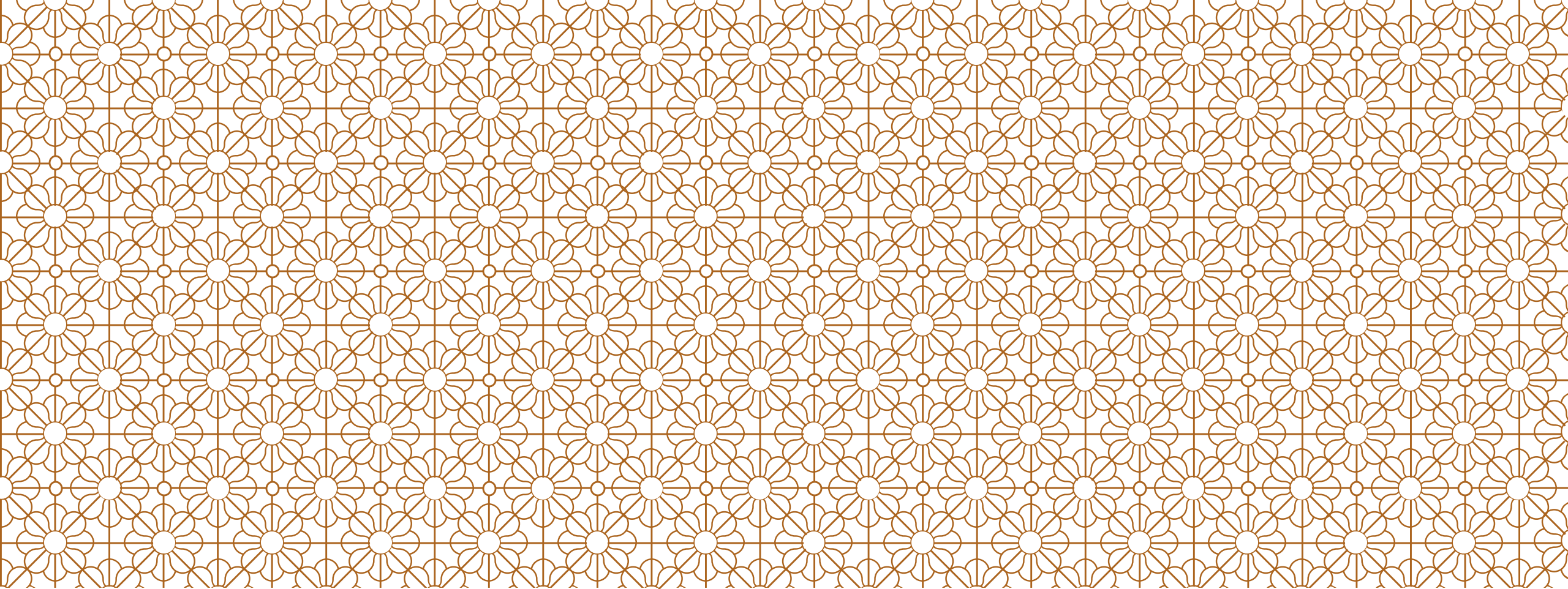
The streamlit web app takes the details of a sample news article as a mini dataset, uses it to predict the legitimacy of the news article using the models trained using the jupyter notebook and returns the probability of the news article being a legitimate one.

Here are the links to the repository and the deployed app:

- [Link to Google Colab](#)
- [Link to GitHub repository](#)
- [Link to the Deployed App \(on Streamlit Cloud\)](#)

CONCLUSION

This project demonstrates a complete fake news detection workflow, from raw data cleaning to model comparison and application-level prediction. By combining structured features with text-based signals, it creates a practical classification system that can distinguish fake and real news articles with machine learning approaches, using both classical and deep learning algorithms.



THANK YOU

Presentation by
Asmit Das Choudhury